

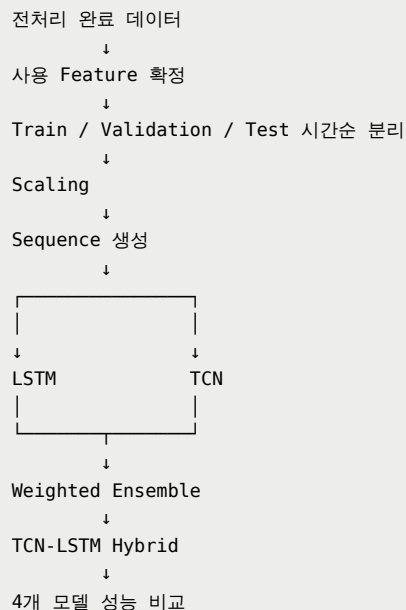
VS Code 구현 가이드 – LSTM·TCN·Weighted Ensemble·TCN-LSTM

이번 구현은 다음 4개 모델을 단계적으로 비교하는 구조로 진행한다.

① LSTM → ② TCN → ③ LSTM-TCN Weighted Ensemble → ④ TCN-LSTM Hybrid

처음부터 복잡하게 만들지 않고, LSTM과 TCN을 각각 정상적으로 학습시키는 것부터 시작한 뒤 결합 모델로 확장한다.

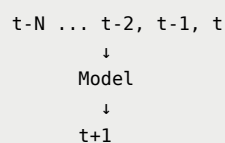
1. 전체 구현 흐름



예측 문제는 다음 기준으로 구현한다.

과거 N시간의 정보를 이용해 다음 1시간의 평균 전력사용량을 예측

즉,



이다.

2. VS Code 프로젝트 구조

프로젝트 폴더는 다음처럼 구성하는 것을 추천한다.

```
LSAXMakers_Project/
├── data/
│   ├── raw/
│   │   └── okm_augumented_2021.csv
│   └── processed/
│       └── okm_cleaned.csv
├── src/
│   ├── preprocessing.py
│   ├── data_pipeline.py
│   └── models/
│       ├── lstm_model.py
│       ├── tcn_model.py
│       └── tcn_lstm_model.py
├── results/
│   ├── models/
│   ├── predictions/
│   └── metrics/
├── train_lstm.py
├── train_tcn.py
├── train_tcn_lstm.py
├── weighted_ensemble.py
└── requirements.txt
```

정현님이 만든

```
src/preprocessing.py
```

는 그대로 두고, 우리는 모델링 파트를 추가한다.

3. 가상환경 만들기

VS Code 터미널을 열고 프로젝트 폴더에서 실행한다.

```
python -m venv .venv
```

Windows에서는:

```
.venv\Scripts\activate
```

정상적으로 실행되면 터미널 앞에 다음처럼 표시된다.

```
(.venv)
```

4. 필요한 라이브러리 설치

```
pip install pandas numpy scikit-learn matplotlib tensorflow joblib optuna
```

설치 후 `requirements.txt` 를 만든다.

```
pip freeze > requirements.txt
```

5. 전처리 완료 데이터 준비

정현님 전처리가 끝난 뒤 최종 데이터는 예를 들어 다음 위치에 저장한다고 가정한다.

```
data/processed/okm_cleaned.csv
```

중요한 점은 **모델링 전에 Feature를 최종 확정해야 한다는 것이다.**

우선 예시 Feature는 다음 정도로 시작한다.

```
전력_평균_실수  
생산량  
기온  
풍속  
습도  
강수량  
시간_sin  
시간_cos  
주말여부
```

여기에 정현님과 합의한 뒤

```
공휴일여부  
영업일여부  
점심시간여부  
가동상태여부
```

등을 추가할 수 있다.

반대로 현재 기준에서는

```
공장인원  
인당생산량
```

은 사용하지 않는 방향을 우선 검토한다.

6. 가장 먼저 `data_pipeline.py` 만들기

경로:

```
src/data_pipeline.py
```

아래 코드를 작성한다.

```
from pathlib import Path
import joblib
import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler

# ===== 기본 설정 =====
DATA_PATH = Path("data/processed/okm_cleaned.csv")
TARGET_COLUMN = "전력_평균실수"
FEATURE_COLUMNS = ["전력_평균실수", "생산량", "기온", "풍속", "습도", "강수량"]
LOOKBACK = 24

# ===== 데이터 불러오기 =====
def load_data():
    df = pd.read_csv(DATA_PATH)
```

7. 시간 순서대로 Train / Validation / Test 나누기

시계열 데이터이므로 절대로

```
train_test_split(shuffle=True)
```

를 사용하지 않는다.

시간순으로 나눈다.

`data_pipeline.py` 아래에 추가한다.

```
def get_split_indices(df):
    train_end = pd.Timestamp("2021-06-30 23:00:00")
    val_end = pd.Timestamp("2021-08-15 23:00:00")
    train_mask = df["datetime"] <= train_end
    val_mask = (df["datetime"] > train_end) & (df["datetime"] <= val_end)
    test_mask = df["datetime"] > val_end
    return train_mask, val_mask, test_mask
```

현재는 예시로

```
Train
1월 ~ 6월

Validation
7월 ~ 8월 15일

Test
8월 16일 ~ 9월 14일
```

로 잡았다.

내일 팀 회의에서 기간은 조정해도 된다.

중요한 원칙은

Test 구간은 모델 선정과 튜닝 과정에서 보지 않는다.

이다.

8. Scaling

LSTM과 TCN 모두 Scaling을 적용한다.

다만 중요한 점이 있다.

Scaler는 Train 데이터로만 학습해야 한다.

Validation과 Test까지 포함해서 Scaler를 학습하면 미래 데이터의 정보를 미리 보는 Leakage가 발생한다.

`data_pipeline.py`에 추가한다.

```
def scale_data(df, train_mask):    X = df[FEATURE_COLUMNS].copy()    y = df[[TARGET_COLUMN]].copy()    x_scaler = StandardScaler()    y_scaler = StandardScaler()    # Train 데이터만 이용해 fit    x_scaler.fit(X.loc[train_mask])    y_scaler.fit(y.loc[train_mask])    # 전체 데이터 transform    X_scaled = x_scaler.transform(X)    y_scaled = y_scaler.transform(y)    return X_scaled, y_scaled, x_scaler, y_scaler
```

9. Sequence 데이터 만들기

이 부분이 LSTM/TCN의 핵심이다.

일반 머신러닝에서는 한 행을 입력하지만,

LSTM과 TCN에서는

과거 24시간
↓
다음 1시간

형태로 데이터를 만들어야 한다.

함수를 추가한다.

```
def create_sequences(X, y, target_indices, lookback=24):    X_seq = []    y_seq = []    indices = []    for i in range(lookback, len(X)):        # 과거 lookback 시간        X_window = X[i-lookback:k:i]        # 다음 시점 Target        y_target = y[i]        X_seq.append(X_window)        y_seq.append(y_target)        indices.append(i)    X_seq = np.array(X_seq)    y_seq = np.array(y_seq)    indices = np.array(indices)    return X_seq, y_seq, indices
```

LOOKBACK이 24라면

1시
2시
3시
...

24시
↓
25시 전력 예측

형태가 된다.

10. 모델별 데이터 분리

추가한다.

```
def split_sequences( X_seq, y_seq, indices, train_mask, val_mask, test_mask): train_array = train_mask.to_numpy() val_array = val_mask.to_numpy() test_array = test_mask.to_numpy() train_idx = train_array[indices] val_idx = val_array[indices] test_idx = test_array[indices] X_train = X_seq[train_idx] y_train = y_seq[train_idx] X_val = X_seq[val_idx] y_val = y_seq[val_idx] X_test = X_seq[test_idx] y_test = y_seq[test_idx] return ( X_train, y_train, X_val, y_val, X_test, y_test, )
```

11. 최종 데이터 준비 함수

마지막으로 한 번에 사용할 수 있게 만든다.

```
def prepare_dataset(lookback=24): df = load_data() train_mask, val_mask, test_mask = get_split_indices(df) X_scaled, y_scaled, x_scaler, y_scaler = scale_data(df, train_mask) X_seq, y_seq, indices = create_sequences(X_scaled, y_scaled, df.index, lookback) ( X_train, y_train, X_val, y_val, X_test, y_test, ) = split_sequences(X_seq, y_seq, indices, train_mask, val_mask, test_mask, ) return { "df": df, "X_train": X_train, "y_train": y_train, "X_val": X_val, "y_val": y_val, "X_test": X_test, "y_test": y_test, "x_scaler": x_scaler, "y_scaler": y_scaler, }
```

12. 먼저 데이터가 제대로 만들어졌는지 확인

프로젝트 최상단에 임시 파일을 하나 만든다.

test_data_pipeline.py

```
from src.data_pipeline import prepare_datasetdata = prepare_dataset(lookback=24)print("X_train:", data["X_train"].shape)print("y_train:", data["y_train"].shape)print("X_val:", data["X_val"].shape)print("y_val:", data["y_val"].shape)print("X_test:", data["X_test"].shape)print("y_test:", data["y_test"].shape)
```

실행:

python test_data_pipeline.py

예를 들어

```
X_train: (4000, 24, 6)
```

처럼 나온다면 의미는 다음과 같다.

4000 = Sequence 개수

24 = 과거 24시간

6 = Feature 개수

13. LSTM 모델 만들기

파일:

```
src/models/lstm_model.py
```

```
from tensorflow.keras import Sequentialfrom tensorflow.keras.layers import ( Input, LSTM, Dense, Dropout,)def build_lstm( input_shape, hidden_units=64, dropout=0.2,): model = Sequential([ Input(shape=input_shape), LSTM(units=hidden_units), Dropout(dropout), Dense(units=32, activation="relu"), Dense(1) ]) model.compile(optimizer="adam", loss="mse", metrics=["mae"], ) return model
```

구조는 단순하다.

```
24시간 Sequence
↓
LSTM
↓
Dropout
↓
Dense
↓
다음 시간 전력
```

처음부터 LSTM Layer를 여러 개 쌓지 않는다.

14. LSTM 학습 파일

파일:

```
train_lstm.py
```

```
from pathlib import PathX_train = data["X_train"]y_train = data["y_train"]X_val = data["X_val"]y_val = data["y_val"]print("Train:", X_train.shape)print("Validation:", X_val.shape)# =====# 모델 생성# =====model = build_lstm( input_shape=
```

```
(X_train.shape[1], X_train.shape[2]), hidden_units=64, dropout=0.2,)model.summary()# ===== Callback# ===== Path("results/models").mkdir(
parents=True, exist_ok=True)early_stopping = EarlyStopping( monitor
="val_loss", patience=10, restore_best_weights=True,)checkpoint = ModelCheckpoint( "results/mod
els/lstm_best.keras", monitor="val_loss", save_best_only=True,)# =====
===== # 학습# ===== history = model.fit( X_train, y_train,
validation_data=( X_val, y_val ), epochs=100, batch_size=32, callbacks=[
early_stopping, checkpoint ], verbose=1,
```

실행:

```
python train_lstm.py
```

15. TCN 모델 만들기

TensorFlow 기본에는 완성된 TCN Layer가 없으므로 **Conv1D**를 이용해 직접 간단한 TCN Block을 만든다.

파일:

```
src/models/tcn_model.py
```

```
from tensorflow.keras import Model, Input, Conv1D, Dense, Dropout, Add, Activation,
GlobalAveragePooling1D,def residual_tcn_block( x, filters, kernel_size, dilation_rate, d
ropout,): shortcut = x x = Conv1D( filters=filters, kernel_size=kernel_size,
dilation_rate=dilation_rate, padding="causal", activation="relu", )(x) x = Dropout(d
ropout)(x) x = Conv1D( filters=filters, kernel_size=kernel_size, dilation_rate=d
ilation_rate, padding="causal", )(x) # Residual shape 맞추기 if shortcut.shape[-1] != fil
ters: shortcut = Conv1D( filters=filters, kernel_size=1, padding
="same", )(shortcut) x = Add()([ x, shortcut ]) x = Activation("relu")(x)
return xdef build_tcn( input_shape, filters=64, kernel_size=3, dropout=0.2,
```

16. TCN의 Dilation 이해

TCN의 핵심이다.

```
dilation = 1
↓
바로 주변 시간 확인

dilation = 2
↓
2시간 간격 확인

dilation = 4
↓
4시간 간격 확인

dilation = 8
↓
8시간 간격 확인
```

즉 일반 CNN보다 더 넓은 과거 범위를 효율적으로 본다.

17. TCN 학습

파일:

train_tcn.py

```
from pathlib import Path
from tensorflow.keras.callbacks import ( EarlyStopping, ModelCheckpoint,)
from src.data_pipeline import prepare_dataset
from src.models.tcn_model import build_tcn

data = prepare_dataset(lookback=24)
X_train = data["X_train"]
y_train = data["y_train"]
X_val = data["X_val"]
y_val = data["y_val"]
model = build_tcn(input_shape=(X_train.shape[1], X_train.shape[2]), filters=64, kernel_size=3, dropout=0.2)
model.summary()
```

실행:

python train_tcn.py

18. 평가 함수 만들기

파일:

src/evaluate.py

```
import numpy as np
from sklearn.metrics import ( mean_absolute_error, mean_squared_error, r2_score, precision_score, recall_score, f1_score,)

def evaluate_regression(y_true, y_pred, peak_threshold=180):
    mae = mean_absolute_error(y_true, y_pred)
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    r2 = r2_score(y_true, y_pred)
    true_peak = (
```

19. Prediction은 반드시 원래 kW 단위로 복원

모델 출력은 Scaling된 값이다.

따라서 평가할 때 반드시

y_scaler.inverse_transform()

을 사용한다.

예:

```
prediction_scaled = model.predict(X_test)
prediction = y_scaler.inverse_transform(prediction_scaled)
actual = y_scaler.inverse_transform(y_test)
```

20. IEEE 2023 방식 Weighted Ensemble 구현

LSTM과 TCN이 모두 학습된 뒤 실행한다.

파일:

weighted_ensemble.py

```
import numpy as np
from tensorflow.keras.models import load_model
from sklearn.metrics import mean_squared_error
from src.data_pipeline import prepare_dataset
from src.evaluate import evaluate_regression

# 데이터
data = prepare_dataset(lookback=24)
X_val = data["X_val"]
y_val = data["y_val"]
X_test = data["X_test"]
y_test = data["y_test"]
y_scaler = data["y_scaler"]

# 모델 로드
lstm = load_model("results/models/lstm_best.keras")
tcn = load_model("results/models/tcn_best.keras")

# Validation Prediction
lstm_val_scaled = lstm.predict(X_val)
tcn_val_scaled = tcn.predict(X_val)
lstm_val = y_scaler.inverse_transform(lstm_val_scaled).flatten()
tcn_val = y_scaler.inverse_transform(tcn_val_scaled).flatten()
y_val_real = y_scaler.inverse_transform(y_val)
```

21. Ensemble Weight 계산

아래를 이어서 작성한다.

```
lstm_inverse_error = 1 / lstm_mse
tcn_inverse_error = 1 / tcn_mse
total = (lstm_inverse_error + tcn_inverse_error)
lstm_weight = (lstm_inverse_error / total)
tcn_weight = (tcn_inverse_error / total)
print("LSTM Weight:", lstm_weight)
print("TCN Weight:", tcn_weight)
```

예를 들어

```
LSTM Weight = 0.42
TCN Weight = 0.58
```

이 나오면

```
최종 예측
=
LSTM × 0.42
+
TCN × 0.58
```

이 된다.

22. Test Ensemble Prediction

```
lstm_test = y_scaler.inverse_transform(lstm.predict(X_test)).flatten()
tcn_test = y_scaler.inverse_transform(tcn.predict(X_test)).flatten()
ensemble_prediction = (lstm_test * lstm_weight + tcn_test * tcn_weight)
y_test_real = y_scaler.inverse_transform(y_test).flatten()
metrics = evaluate_regression(y_test_real, ensemble_prediction)
print(metrics)
```

23. IET 2026 기반 TCN-LSTM Hybrid

이번에는 Ensemble이 아니라 하나의 **Neural Network** 안에서 TCN과 LSTM을 연결한다.

파일:

src/models/tcn_lstm_model.py

```
from tensorflow.keras import Model
from tensorflow.keras.layers import (Input, Conv1D, LSTM, Dense, Dropout, Add, Activation)

def residual_block(x, filters, kernel_size, dilation_rate, dropout):
    shortcut = x
    x = Conv1D(filters=filters, kernel_size=kernel_size, dilation_rate=dilation_rate, padding="causal", activation="relu")(x)
    x = Dropout(dropout)(x)
    x = Conv1D(filters=filters, kernel_size=kernel_size, dilation_rate=dilation_rate, padding="causal", activation="relu")(x)
    x = Add()(shortcut, x)
    return x
```

구조는 다음과 같다.

```
과거 24시간
↓
TCN Block
↓
TCN Block
↓
TCN Block
↓
Temporal Feature
↓
LSTM
↓
Dense
↓
t+1 전력
```

24. TCN-LSTM 학습

파일:

train_tcn_lstm.py

```
from pathlib import Path
from tensorflow.keras.callbacks import (EarlyStopping, ModelCheckpoint,)
from src.data_pipeline import prepare_dataset
from src.models.tcn_lstm_model import build_tcn_lstm

def train():
    dataset = prepare_dataset()
    model = build_tcn_lstm()
    model.compile(optimizer=adam, loss=loss)
    model.fit(dataset, callbacks=[EarlyStopping, ModelCheckpoint])
```

```
ta = prepare_dataset(lookback=24)X_train = data["X_train"]y_train = data["y_train"]X_val = data["X_val"]y_val = data["y_val"]model = build_tcn_lstm(input_shape=(X_train.shape[1], X_train.shape[2]), filters=64, lstm_units=64, kernel_size=3,
```

25. 처음부터 파라미터 튜닝하지 않는다

처음에는 전부 동일한 조건으로 실행한다.

공통 초기 조건

```
Lookback = 24

Batch Size = 32

Epoch = 100

Learning Rate = 기본 Adam

Dropout = 0.2

Hidden / Filters = 64

EarlyStopping patience = 10
```

먼저 기본 모델이 정상적으로 돌아가는지 확인한다.

26. 1차 결과 비교

가장 먼저 아래 네 모델의 기본 성능표를 만든다.

Model	RMSE	MAE	R ²	Peak Recall	Peak F1
LSTM					
TCN					
Weighted Ensemble					
TCN-LSTM					

여기서 가능성이 있는 모델만 튜닝한다.

27. 가장 먼저 튜닝할 값

이번 데이터에서 가장 먼저 봐야 할 것은 **Lookback**이다.

```
24시간
48시간
72시간
168시간
```

특히 현재 EDA에서는

```
Lag 1h ≈ 0.90  
Lag 168h ≈ 0.74
```

가 확인됐기 때문에 168은 반드시 실험해볼 가치가 있다.

28. LSTM 튜닝 범위

```
lookback  
24 / 48 / 72 / 168  
  
lstm_units  
32 / 64 / 128  
  
dropout  
0.0 / 0.1 / 0.2 / 0.3  
  
batch_size  
16 / 32 / 64  
  
learning_rate  
0.0001 / 0.0003 / 0.001  
  
layer  
1 / 2
```

29. TCN 튜닝 범위

```
lookback  
24 / 48 / 72 / 168  
  
filters  
32 / 64 / 128  
  
kernel_size  
2 / 3 / 5  
  
dropout  
0.0 / 0.1 / 0.2 / 0.3  
  
dilation  
[1,2,4,8]  
[1,2,4,8,16]  
  
learning_rate  
0.0001 / 0.0003 / 0.001
```

30. TCN-LSTM 튜닝 범위

```
lookback
24 / 48 / 72 / 168

TCN filters
32 / 64

LSTM units
32 / 64 / 128

kernel size
2 / 3 / 5

dropout
0.1 / 0.2 / 0.3

learning rate
0.0001 / 0.0003 / 0.001
```

Hybrid는 파라미터가 많기 때문에 LSTM과 TCN 최적값을 먼저 찾은 후 범위를 좁히는 것이 좋다.

31. 실제 실행 순서

VS Code에서는 아래 순서대로 진행한다.

STEP 1

정현님 전처리 완료

```
okm_cleaned.csv
```

확보

STEP 2

```
python test_data_pipeline.py
```

데이터 Shape 확인

STEP 3

```
python train_lstm.py
```

STEP 4

```
python train_tcn.py
```

STEP 5

```
python weighted_ensemble.py
```

STEP 6

```
python train_tcn_lstm.py
```

STEP 7

4개 기본모델 성능 비교

STEP 8

상위 모델만 Hyperparameter Tuning

32. 가장 중요한 주의사항

이번 프로젝트에서 모델 구조보다 더 중요할 수 있는 부분이다.

① 시간순서를 절대 섞지 않는다.

과거 → 미래

순서를 유지한다.

② Scaler는 Train 데이터로만 Fit한다.

```
Train
↓
Scaler Fit

Validation/Test
↓
Transform만
```

③ 미래의 전력값을 Feature에 넣지 않는다.

특히

15분
30분
45분
60분

과 Target의 시간 관계를 반드시 확인한다.

④ Test는 마지막까지 보지 않는다.

모델 선택:

Validation

최종 평가:

Test

⑤ LSTM과 TCN은 반드시 동일한 데이터로 비교한다.

동일 Train
동일 Validation
동일 Test
동일 Feature
동일 Target
동일 평가 Metric

그래야 모델간 비교가 공정하다.

최종 구현 전략

이번 프로젝트의 구현 순서는 다음과 같이 정리한다.

```
[Data]
정현님 전처리
↓
Sequence Dataset

↓

[Base Models]

LSTM
TCN

↓

[Paper-based Model 1]

IEEE EECR 2023
Weighted LSTM-TCN Ensemble

↓

[Paper-based Model 2]

IET 2026
TCN → LSTM Hybrid

↓

[Evaluation]

RMSE
MAE
R2
Peak Recall
```

```
Peak F1
↓
[Hyperparameter Tuning]
Lookback
Hidden Units
Filters
Kernel Size
Dropout
Learning Rate
Batch Size
↓
[Final Model]
최종 전력수요 예측
+
Peak 위험 분석
```

특히 **내일 바로 해야 할 일은 LSTM이나 TCN 코드를 먼저 고치는 것이 아니라 `data_pipeline.py` 가 정확하게 동작하게 만드는 것입니다.** 데이터 입력 Shape, Target의 `t+1` 이동, Train/Val/Test 분리, Scaling이 정확해야 그 뒤 LSTM·TCN·Hybrid 결과도 신뢰할 수 있습니다.

전처리 최종본이 나오면 위 코드에서 **실제 컬럼명을 기준으로 `FEATURE_COLUMNS` 와 Target을** 확인한 뒤, 그 상태에서 LSTM → TCN 순서로 실행하면 됩니다.